

EduHackers Research Phase — Recovered History

Purpose

This file reconstructs the key research decisions that shaped Vossie before the EduHackers 2.0 consolidation.

Research framing

The project was researched as more than an e-commerce app. The engineering and product research examined marketplace matching, Uber-style dispatch, campus hubs, seller/courier workflows, identity, payments, realtime state, custody, Incubation and cross-campus operations.

Two application families

A major research decision was to separate the student marketplace experience from staff/operations workflows while keeping a shared backend.

Marketplace family

- Buyer
- Seller
- Courier

Operations family

- Hub Operator
- Transfer/Transport Operator
- Incubation
- Moderator
- Finance
- Support
- Admin

This prevents a student-facing app from becoming cluttered with dense staff tables and prevents staff users from being given inappropriate marketplace screens or permissions.

Identity research

The research rejected format checks as authentication. A student number beginning with EDUV or an institutional email domain may be useful validation metadata, but neither proves identity.

The target model became institutional federation:

Vossie → Eduvos/Microsoft sign-in → existing MFA/Authenticator → signed identity result → active-student eligibility check → Vossie session.

Vossie should not receive the institutional password, Authenticator seed or MFA approval secret.

Progressive roles

The research established that a student should have one identity and progressively activate capabilities:

- Buyer: institutional/student verification.
- Seller: Buyer verification + additional seller onboarding and payout/compliance setup.
- Courier: verified identity + courier onboarding, transport mode and payout setup.
- Staff: assigned membership, stronger MFA/role controls and auditability.

Backend authority

The UI can hide or show actions, but the server remains authoritative. Client-side role selection never grants permission.

Marketplace and dispatch research

Seller matching and courier dispatch should use explicit state machines and atomic acceptance. If two users accept the same opportunity at the same time, the database must guarantee only one successful lock.

Hub and custody research

Packages become auditable objects. QR scanning changes custody only after server validation. Hub receipt, courier pickup and buyer handover must be traceable.

Financial research

The system should not mutate a single wallet balance blindly. Financial movements should be append-only ledger entries with provider references and idempotency keys.

Realtime research

REST/transactional APIs remain the source of truth. WebSockets/realtime are used for fast updates such as courier location, seller offers, package scans and order states. After reconnect, the client fetches authoritative state again.

Platform research stack

Research candidates converged around TypeScript, Next.js/React, React Native/Expo for native mobile when needed, Tauri for possible desktop packaging, PostgreSQL/PostGIS, realtime infrastructure and GitHub-based engineering workflow.

Research outcome

The work moved from “marketplace concept” into an implementation system with detailed contracts, screens, APIs, tables, acceptance criteria and release gates.